Circuit Design with Logic Gates

Logic Gates

- Logic gates are the basic building blocks of any digital system.
- It is an electronic circuit having one or more than one input and only one output.
- The relationship between the input and the output is based on a **certain logic**. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.
- Most electronic devices we use today will have some form of logic gates in them. For example, logic gates can be used in technologies such as smartphones, tablets or within memory devices.

Logic Gates

- At any one time, a digital device will be in one of these two binary situations.
- A light bulb can be used to demonstrate the operation of a logic gate.
- When logic 0 is supplied to the switch, it is turned off, and the bulb does not light up.
- The switch is in an ON state when logic 1 is applied, and the bulb would light up.
- In integrated circuits (IC), logic gates are widely employed..

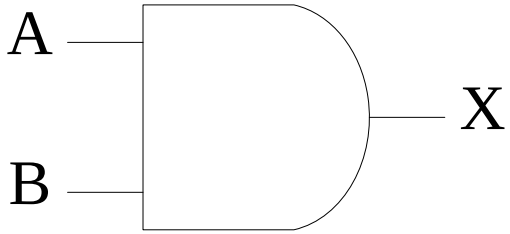
A Simple Circuit

A	X
0	0
1	1

$$X = A$$

AND Gate

- An AND gate has a single output and two or more inputs.
- When all of the inputs are 1, the output of this gate is 1.
- The AND gate's Boolean logic is **Y=A.B** if there are two inputs A and B.
- Therefore, in And gate, the output is high when all the inputs are high.

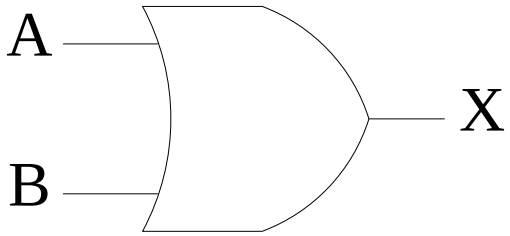


$$X = A.B$$

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

- Two or more inputs and one output can be used in an OR gate.
- The logic of this gate is that if at least one of the inputs is 1, the output will be 1.
- The OR gate's output will be given by the following mathematical procedure if there are two inputs A and B: Y=A+B
- Therefore, in the OR gate the output is high when any of the inputs is high.

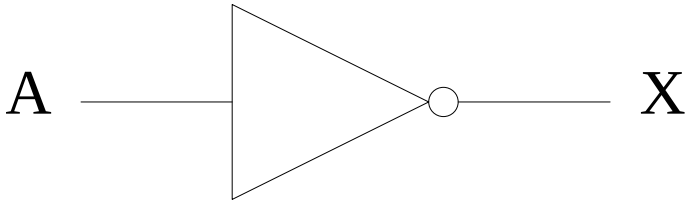


$$X = A + B$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate

- NOT gate is also known as **Inverter**. It has one input A and one output Y.
- The NOT gate is a basic one-input, one-output gate.
- When the input is 1, the output is 0 and vice versa. A NOT gate is sometimes called as an inverter because of its feature.
- If there is only one input A, the output may be calculated using the Boolean equation Y=A'.
- A NOT gate, as its truth table shows, reverses the input signal.

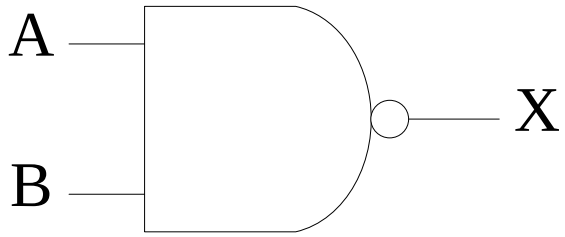


$$X = \overline{A}$$

A	X
0	1
1	0

NAND Gate

- A NAND gate, sometimes known as a 'NOT-AND' gate, is essentially a Not gate followed by an AND gate.
- This gate's output is 1 only if none of the inputs is 1. Alternatively, when all of the inputs are not high and at least one is low, the output is high.
- If there are two inputs A and B, the Boolean expression for the NAND gate is Y=(A.B)'
- The NAND gate is known as a universal gate because it may be used to implement the AND, OR, and NOT gates.

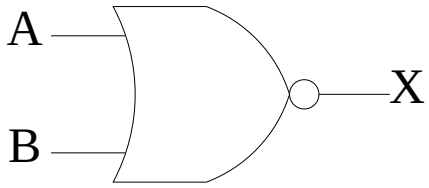


$$X = \overline{A.B}$$

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

- A NOR gate, sometimes known as a “NOT-OR” gate, consists of an OR gate followed by a NOT gate.
- This gate’s output is 1 only when all of its inputs are 0. Alternatively, when all of the inputs are low, the output is high.
- The Boolean statement for the NOR gate is Y=(A+B)’ if there are two inputs A and B.
- The NOR gate is sometimes known as a universal gate since it may be used to implement the OR, AND, and NOT gates.

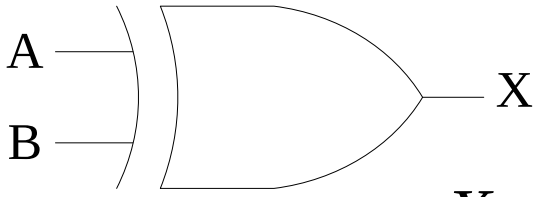


$$X = \overline{A + B}$$

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

XOR (eXclusive OR**) Gate**

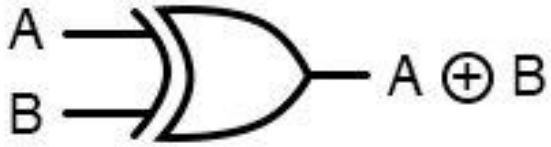
- XOR or Ex-OR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-OR gate is abbreviated as EX-OR gate or sometime as X-OR gate. It has n input (n >= 2) and one output.
- The Exclusive-OR or 'Ex-OR' gate is a digital logic gate that accepts more than two inputs but only outputs one value.
- If any of the inputs is 'High,' the output of the XOR Gate is 'High.' If both inputs are 'High,' the output is 'Low.' If both inputs are 'Low,' the output is 'Low.'
- The Boolean equation for the XOR gate is Y=A'.B+A.B' if there are two inputs A and B.



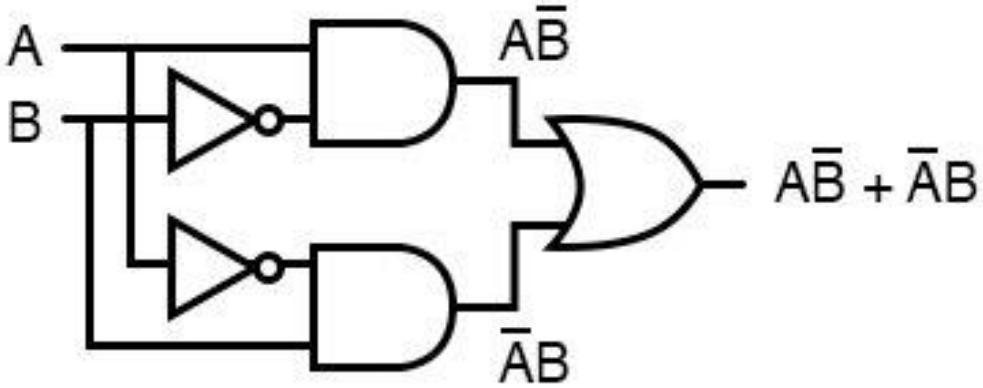
$$X = A \oplus B$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

XOR (eXclusive OR**) Gate**



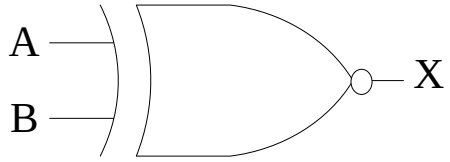
. . . is equivalent to . .



$$A \oplus B = A\bar{B} + \bar{A}B$$

XNOR Gate

- XNOR gate is a special type of gate. It can be used in the half adder, full adder and subtractor. The exclusive-NOR gate is abbreviated as EX-NOR gate or sometime as X-NOR gate. It has n input (n >= 2) and one output. The XNOR gate is the complement of the XOR gate.
- The Exclusive-NOR or 'EX-NOR' gate is a digital logic gate that accepts more than two inputs but only outputs one.
- If both inputs are 'High,' the output of the XNOR Gate is 'High.' If both inputs are 'Low,' the output is 'High.' If one of the inputs is 'Low,' the output is 'Low.'
- If there are two inputs A and B, then the XNOR gate's Boolean equation is: Y=A.B+A'B'.



$$X = \overline{A \oplus B}$$

A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

- **Uses of Logic Gates**
- Logic gates are utilized in a variety of technologies. These are components of chips (ICs), which are components of computers, phones, laptops, and other electronic devices.
- Logic gates may be combined in a variety of ways, and a million of these combinations are necessary to make the newest gadgets, satellites, and even robots.
- Simple logic gate combinations can also be found in alarms, buzzers, switches, and street lights. Because these gates can make a choice to start or stop based on logic, they are often used in a variety of sectors.
- Logic gates are also important in data transport, calculation, and data processing. Even transistor-transistor logic and CMOS circuitry make extensive use of logic gates.

⚙️ Main Types of Logic Gates:

Gate	Symbol	Operation	Example Rule
AND	·	Output is 1 only if all inputs are 1	A · B
OR	+	Output is 1 if any input is 1	A + B
NOT	— (bar)	Inverts the input	$\bar{A}$
NAND	AND + NOT	Opposite of AND	(A · B)'
NOR	OR + NOT	Opposite of OR	(A + B)'
XOR	Exclusive OR	Output is 1 if inputs are different	A ⊕ B
XNOR	Exclusive NOR	Output is 1 if inputs are same	(A ⊕ B)'

Boolean Algebra

- The logical symbol 0 and 1 are used for representing the digital input or output.
- The symbols "1" and "0" can also be used for a permanently open and closed digital circuit.
- The digital circuit can be made up of several logic gates.
- To perform the logical operation with minimum logic gates, a set of rules were invented, known as the **Laws of Boolean Algebra**.
- These rules are used to reduce the number of logic gates for performing logic operations.
- The Boolean algebra is mainly used for simplifying and analyzing the complex Boolean expression.
- It is also known as **Binary algebra** because we only use binary numbers in this.
- **George Boole** developed the binary algebra in **1854**.

Rules of Boolean Algebra

- Variable used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
- Complement of a variable is represented by an overbar (-). Thus, complement of variable B is represented as $\overline{B}$ thus if B = 0 then $\overline{B}$ =1 and B = 1 then $\overline{B}$ = 0.
- ORing of the variables is represented by a plus (+) sign between them. For example ORing of A, B, C is represented as A + B + C.
- Logical ANDing of the two or more variable is represented by writing a dot between them such as A.B.C. Sometime the dot may be omitted like ABC.
- There are six types of Boolean Laws.

Boolean Algebra

- **OR RULES (ADDITION)**

- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 1
- A + 0 = A
- A + 1 = 1
- A + A (bar) =1

Boolean Algebra

- **AND RULES (MULTIPLICATION)**

- 0.0 = 0
- 0.1 = 0
- 1.0 = 0
- 1.1 = 1
- A.1 = A
- A.0 = 0
- A.A = A
- A.A (bar) = 0

Commutative law

- Any binary operation which satisfies the following expression is referred to as commutative operation.

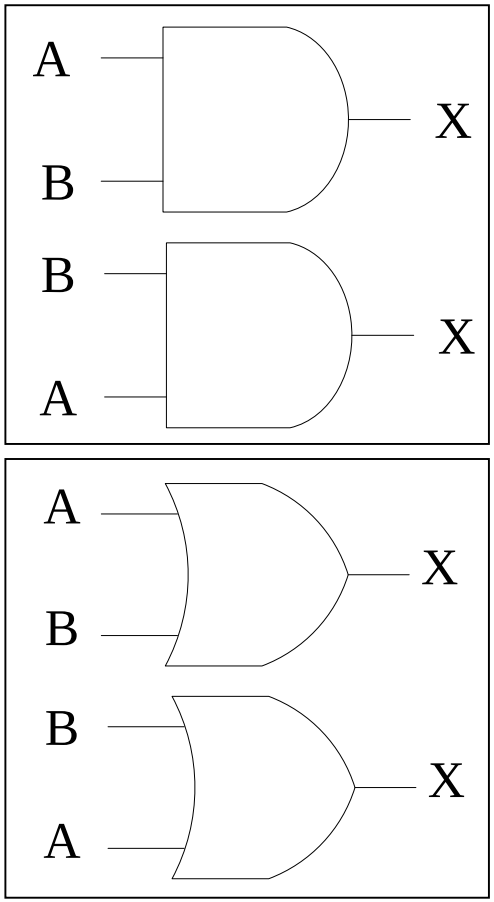
(i) $A.B = B.A$ (ii) $A + B = B + A$

- Commutative law states that changing the sequence of the variables does not have any effect on the output of a logic circuit.

Commutative Law

$$AB = BA$$

$$A + B = B + A$$

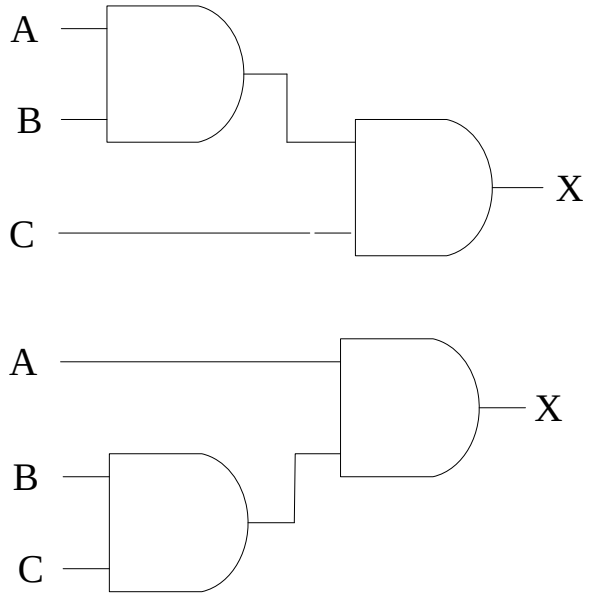


Associative Law

This law states that the order in which the logic operations are performed is irrelevant as their effect is the same.

(i) $(A.B).C = A.(B.C)$

(ii) $(A+B)+C = A+(B+C)$



Distributive Law

This law allows us to open up of brackets. Simply, we can open the brackets in the Boolean expressions.

$$A(B+C) = AB + AC$$

$$A + (BC) = (A+B).(A+C)$$

AND Law

These laws use the AND operation. Therefore they are called as **AND** laws.

(i) $A.0 = 0$

(ii) $A.1 = A$

(iii) $A.A = A$

(iv) $A.\overline{A} = 0$

OR Law

These laws use the OR operation. Therefore they are called as **OR** laws.

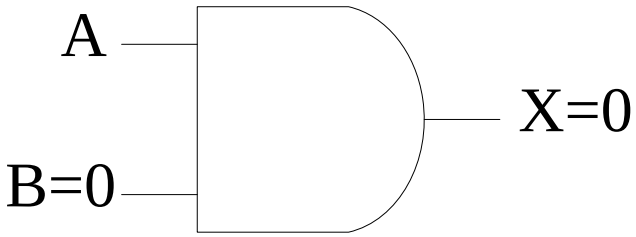
(i) $A + 0 = A$

(ii) $A + 1 = 1$

(iii) $A + A = A$

(iv) $A + \overline{A} = 1$

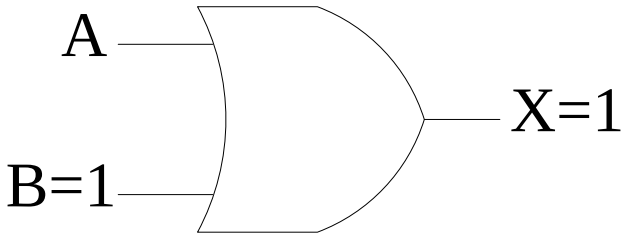
Null Law (AND)



$$A.0 = 0$$

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

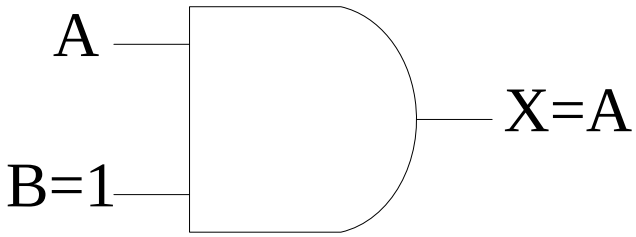
Null Law (OR)



$$A+1=1$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

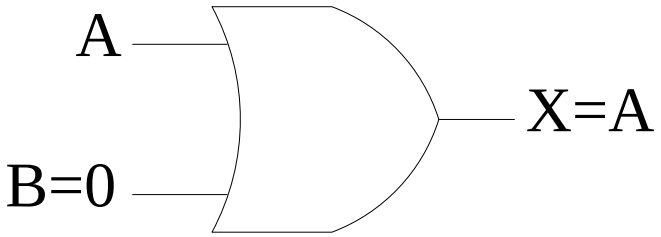
Identity Law (AND)



$$A.1 = A$$

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

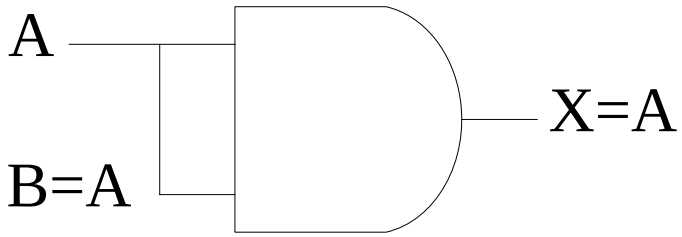
Identity Law (OR)



$$A + 0 = A$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

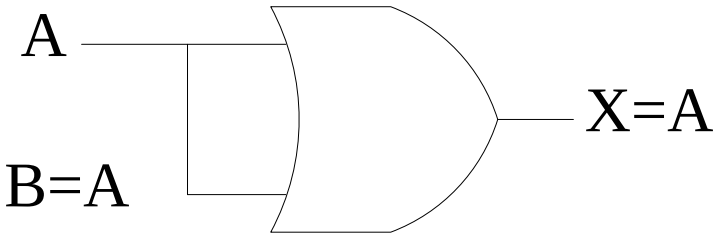
Idempotent Law (AND)



$$A.A = A$$

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

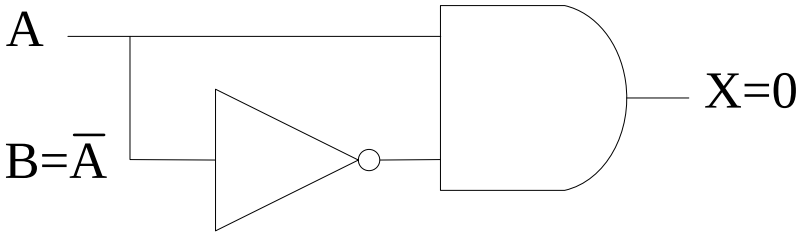
Idempotent Law (OR)



$$A + A = A$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

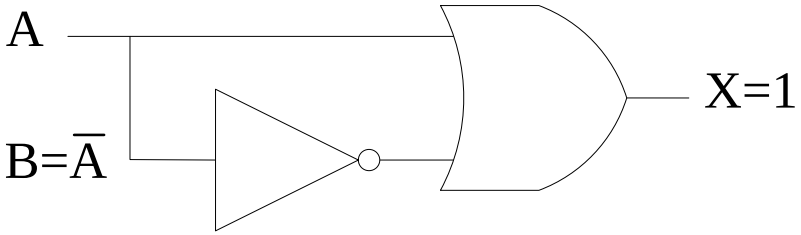
Inverse Law (AND)



$$A.\overline{A} = 0$$

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Inverse Law (OR)



$$A + \overline{A} = 1$$

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Absorption Law

- This law enables a reduction in a complicated expression to a simpler one by absorbing like terms.

A + (A.B) = A (OR Absorption Law)

A + AB

=A(1+B) → since 1 + B = 1

= A.1

= A

Absorption Law

- **A (A + B) = A**

A (A + B) = A.A + A.B

= A+AB → since A . A = A

= A (1 + B)

= A.1

= A

Absorption Law

- **$A + \bar{A}B = A + B$**

$A + \bar{A}B = (A + \bar{A})\,(A + B)$ → since $A+BC = (A+B)(A+C)$ using distributive law

$= 1\,.(A + B)$ → since $A + \bar{A} = 1$

$= A + B$

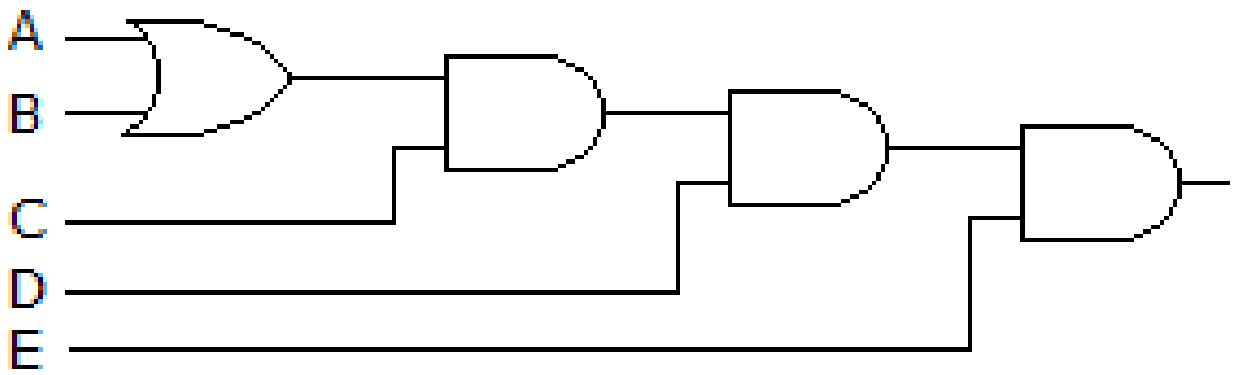
$A\,.\,(\bar{A}+B) = AB$

$= A\,.\,(\bar{A} + B) = A.\,\bar{A} + AB$

$= AB$ → since $A\,\bar{A} = 0$

Examples

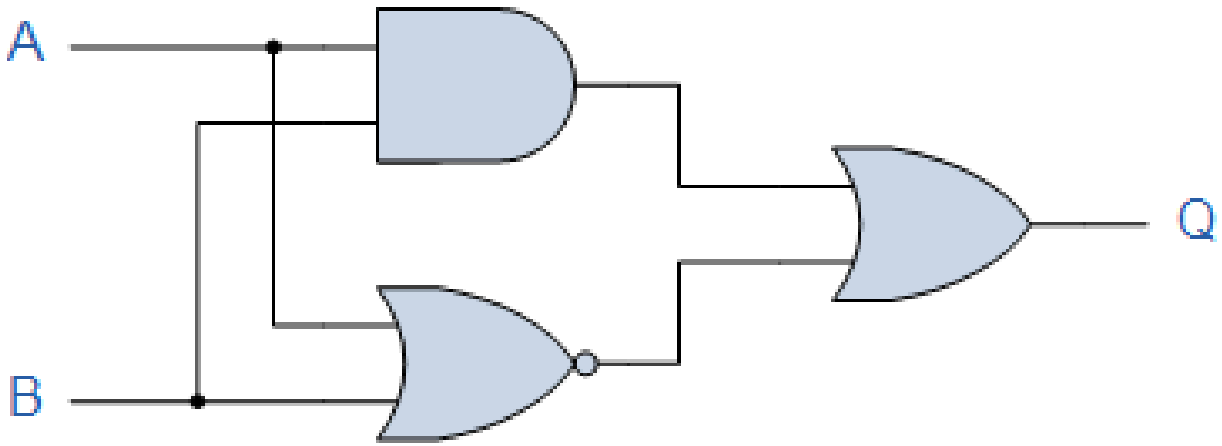
- Derive the Boolean expression for the logic circuit shown below:



$$C(A + B)DE$$

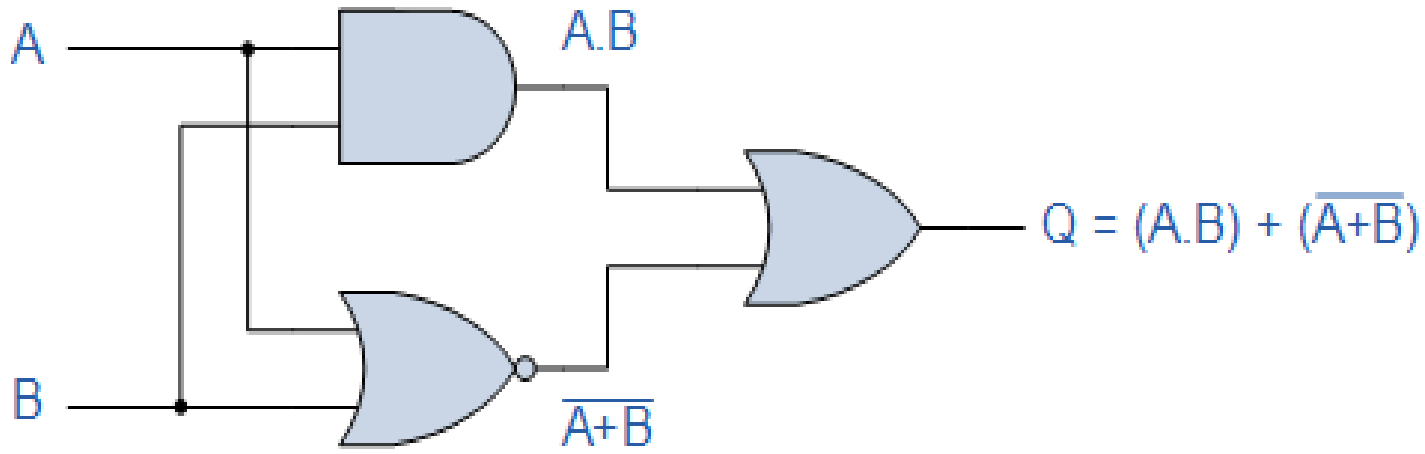
Examples

- Find the Boolean expression for the following system.



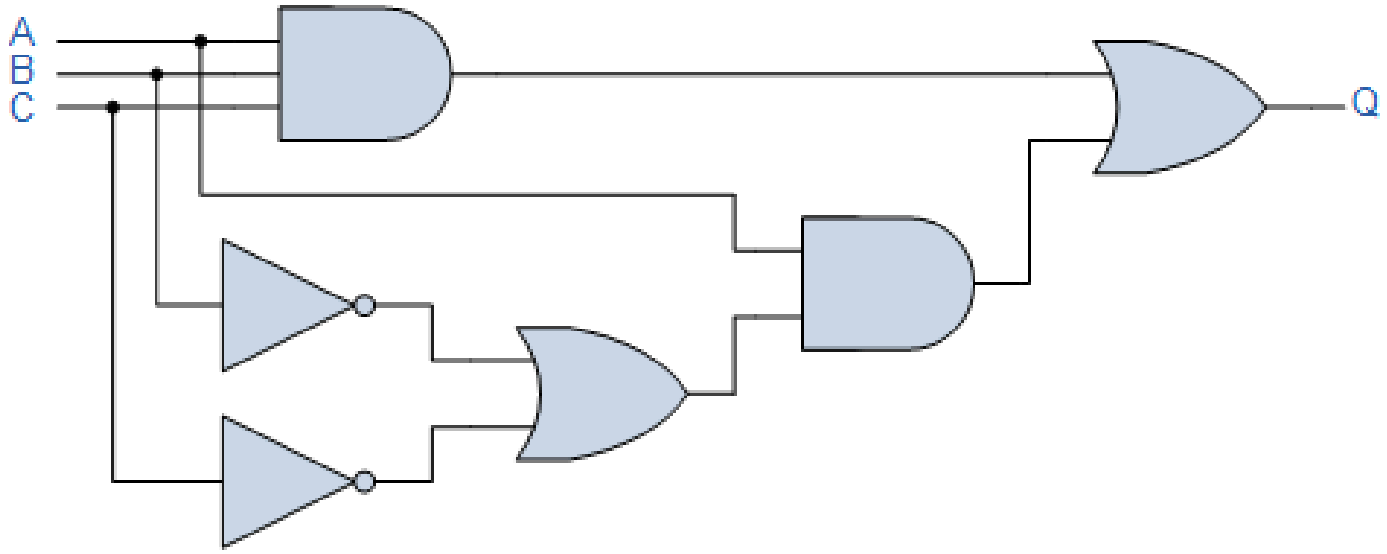
Examples

- Find the Boolean expression for the following system.



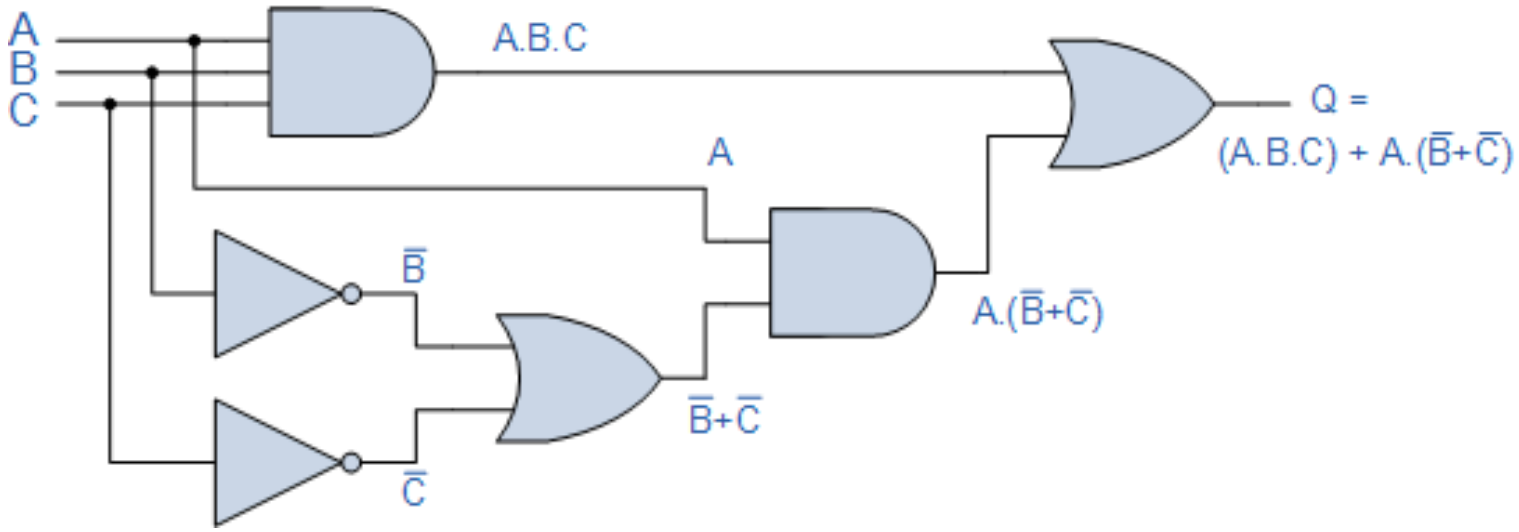
Examples

- Find the Boolean expression for the following system.



Examples

- Find the Boolean expression for the following system.

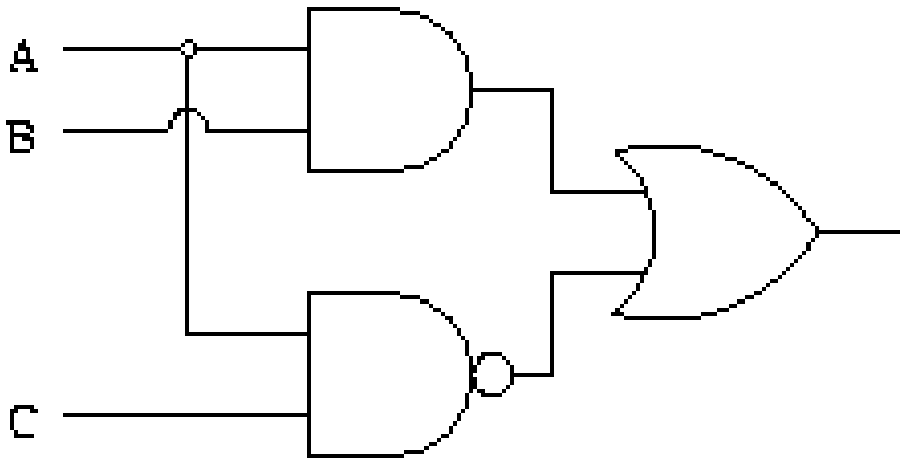


Examples

- Draw logic circuit based on following combination of gates for the expression :
- xy+xy’+y’z
- (A+B)(B+C)
- A’B+(B+C)’
- (AB+C)D
- (A’+B).(A+B’)
- A+BC+D’
- AB+(AC)’

Examples

- Draw logic circuit based on following combination of gates for the expression :
- AB+(AC)’



Examples

- Draw logic circuit based on following combination of gates for the expression :
- A + BC + D’